COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/2026	**DATE SUBMITTED:**	10/16/2026
NAME:	Macasaet, John David Milan, Kherwin Emmanuel Moradillo, Eunice	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

CODE:

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

OUTPUT:

```
OB }
Printing integer: 10
Printing string: T.I.P.
PS C:\Users\TIPQC\Downloads\h
```

Analysis

This program operates through a single thread that executes a single task.Using std::thread t1() this allows to create and start the thread and t1.join() ensures the main thread waits for the new thread to finish before exiting.

Table 13-1. Simple One-Threaded Example

Screenshot	CODE: OUTPUT:

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
   std::string msg = std::to_string(n) + " : " + str;
   std::cout << msg << std::endl;
}
int main() {
    std::vector<std::string> s = {
      "T.I.P.",
      "Competent",
      "Computer",
      "Engineers"
    };
    std::vector<std::thread> threads;
    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }
    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

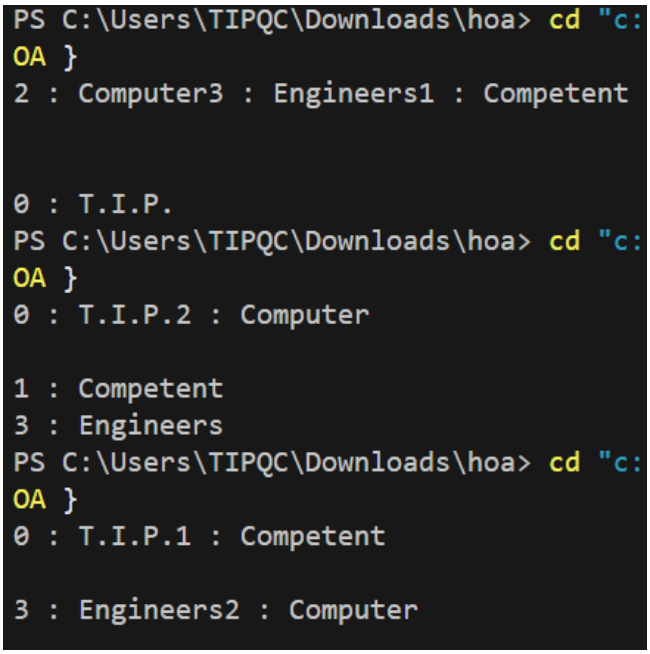
	
Analysis	The output was different every time we ran it. The 4 messages always showed up, but the order changed because the threads run at the same time and we can't know which one finishes first.

Table 13-2. Multithreaded Example.

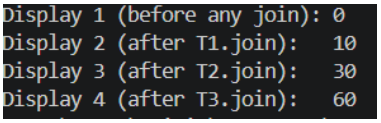
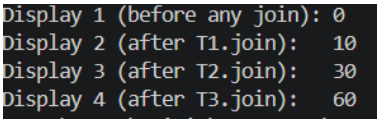
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PART A	
Macasaet	1. **Write a definition of multithreading and its advantages/disadvantages**. - **Multithreading is a technique used in C++ to make multiple parts of the program execute concurrently. An advantage of multithreading is that there is better performance since you are splitting tasks instead of executing code one by one.** 2. **Rationalize the use of multithreading by providing at least 3 use-cases.** - **Multithreading can be used to make performance better by splitting tasks to cut down execution time. Another use is having I/O tasks and network tasks running in the background thread so that the main thread does not get interrupted and be free to run. Multithreading can also be used for UI responsiveness where the heavy background computation get offloaded away from the main thread which handles user interface.** 3. **Differentiate between parallelism and concurrency.** - **Concurrency is the ability of the program to deal with multiple tasks at the same time by making progress over overlapping time windows. On the other hand, parallelism is all about execution wherein the program performs tasks at the exact same instant.**

Milan	1. **Write a definition of multithreading and its advantages/disadvantages**. - Multithreading is when a program makes more than one thread so different tasks can run at the same time. Its advantages are It uses more cpu cores so the program can be faster, and it stays responsive because one thread can wait while the others keep working. While its disadvantages are Data races can happen when threads use the same variable, and the order of results is unpredictable. **2. Rationalize the use of multithreading by providing at least 3 use-cases.** - Web servers can use one thread for each client request, so many users are served at once. Second is, an app with a user interface can load or save files in a background thread while the main thread keeps the screen responsive. **3. Differentiate between parallelism and concurrency.** - Concurrency is when tasks overlap and can share the same memory, and the threads read information from the previous threads. While, parallelism is when copies of the same program run at the same time on different data, and they don't need to talk to each other.
Moradillo	1. **Write a definition of multithreading and its advantages/disadvantages**. - The definition of multithreading is that it's a method or technique where it divides into different tasks so it will run at the same time or concurrently. However, this process has its advantages and disadvantages. The benefits of it is that it can have resource sharing and improved performance and responsiveness. The downside of multithreading is that of debugging complexity and the risk of resource conflicts. **2. Rationalize the use of multithreading by providing at least 3 use-cases.** - The use of multithreading enables performance enhancement and cost efficiency, which is best for web servers that can handle multiple user requests at the same time, data processing that uses parallelism computation for large data sets, and game developments where different codes run at the same time. **3. Differentiate between parallelism and concurrency.** - The difference between parallelism and concurrency is that parallelism can execute multiple tasks at the same time on different processing units while concurrency makes the task progress over time during overlapping time periods on a single processing unit.

PART B

Macasaet	  The output shows display 1 having the lowest since none joined yet. As more threads join the number increases, this results in T3 having the highest number. The numbers confirms which thread joined after the previous one.
Milan	  T1 had the smallest workload and finished first, followed by T2 and then T3, so the global variable grew in steps as each thread was joined. The final value of 60 (10 + 20 + 30) shows that all three add() calls completed before the program ended.

Moradillo

```
Display 1 (before any join): 0
Display 2 (after T1.join):   10
Display 3 (after T2.join):   30
Display 4 (after T3.join):   60
```

```cpp
#include <iostream>
#include <thread>

int globalVar = 0;

void add(int value) {
    for (volatile long i = 0; i < value * 50000000L; i++) { }   // simulated w
    globalVar += value;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Display 1 (before any join): " << globalVar << std::endl;
    T1.join();
    std::cout << "Display 2 (after T1.join):   " << globalVar << std::endl;
    T2.join();
    std::cout << "Display 3 (after T2.join):   " << globalVar << std::endl;
    T3.join();
    std::cout << "Display 4 (after T3.join):   " << globalVar << std::endl;

    return 0;
}
```

The threads started running as soon as they were created, so Display 1 showed 0 because none of them had finished their work yet. Each join() made the main thread wait for one thread, which is why the value rose to 10, then 30, then 60 in the later displays.

PART C

```
ry_partC.cpp -o supplementary_partC } ; if ($?
Unsorted Array:
46 99 25 17 1 77 47 1 39 65

----------------------------

Sorting started concurrently...

Selection Sort finished in 0.0014 ms
Bubble Sort finished in 0.0017 ms
Insertion Sort finished in 0.0007 ms

========== SORTED ARRAYS ==========

Bubble Sort Result:
1 1 17 25 39 46 47 65 77 99

Selection Sort Result:
1 1 17 25 39 46 77 47 65 99

Insertion Sort Result:
1 1 17 25 39 46 47 65 77 99

========== FASTEST ALGORITHM ==========
Insertion Sort was the fastest!
PS C:\Users\LENOVO\Downloads\HOA 7.1>
```

```
ry_partC.cpp -o supplementary_partC } ; if ($?)
Unsorted Array:
51 41 65 50 94 94 10 4 1 36

----------------------------

Sorting started concurrently...

Bubble Sort finished in 0.0015 ms
Insertion Sort finished in 0.0007 ms
Selection Sort finished in 0.0013 ms

========== SORTED ARRAYS ==========

Bubble Sort Result:
1 4 10 36 41 50 51 65 94 94

Selection Sort Result:
1 4 10 36 41 50 51 65 94 94

Insertion Sort Result:
1 4 10 36 41 50 51 65 94 94

========== FASTEST ALGORITHM ==========
Insertion Sort was the fastest!
PS C:\Users\LENOVO\Downloads\HOA 7.1>
```

Macasaet	In the two outputs we see above we can see the time it takes for each sorting algorithm to finish and all three of them are running concurrently. Notice that insertion sort is the fastest among the sorting algorithms, this is because of its nature and how insertion sort works, it inserts each element into its correct position in the already sorted portion of the array. It simply has fewer operations to do compared to bubble or selection sort.
Milan	we see above we can see the time it takes for each sorting algorithm to finish and all three of them are running at the same time in their own threads. Notice that insertion sort is the fastest among the sorting algorithms, this is because the array is small and insertion sort just puts each number in its place in the part that is already sorted.
Moradillo	The three sorting algorithms started together, and each one reported its own time when it finished. Insertion sort had the shortest time in both runs, while bubble sort and selection sort swapped places, so the finishing order of the threads is not always the same.

CONCLUSION	
Macasaet	In this activity we learned about multithreading which allows for the program to run tasks concurrently. This is very useful in reducing execution time. Through this activity, we knew the implementation of multithreading and its advantages and disadvantages.
Milan	In this activity I learned that multithreading lets a program run many tasks at the same time. The output can change every run because we can't know which thread finishes first. In the sorting part, all three sorts gave the same result and insertion sort was the fastest.
Moradillo	In this activity ,I learned how to run multiple tasks at the same time using threads and how to make the main program wait for them to finish. I saw that the results can come out in a different order each time because we can't control which thread finishes first. We think we did well in this activity, but we still need to improve at predicting how threads behave and at fixing problems when they happen.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.